

CYBER WAR ROOM

Attack & Defense Web Security Competition

ENGINEER'S CODE • OFFICIAL RULEBOOK

BUILD & SECURE
60 MINUTES

ATTACK
60 MINUTES

PRESENTATION
5 MINUTES / TEAM

THE CHALLENGE

Build a real web application. Secure it. Package it. Then face a randomly assigned opponent's application and discover, exploit, chain, document, and prove its weaknesses.

Important: Teams may prepare their application before the event. The required application category/theme is announced on contest day. This rulebook does not publish the categories in advance.

1. Team Eligibility

- Each team consists of **4 registered students**.
- Students from any academic year may form a team.
- **One student may belong to one team only.** A participant cannot be registered on multiple teams.
- All team members are expected to participate in the competition and final presentation.

2. What Teams Must Build

- Each team must bring a **functional web application** to the competition.
- Teams may prepare their application before the event.
- The exact application category/theme will be **announced on contest day**; categories are intentionally not disclosed in advance.
- The application must contain meaningful, testable functionality appropriate to the announced category.
- Applications should provide a real attack surface, such as authentication, user/admin areas, CRUD operations, search, APIs, file handling, database interactions, business logic, or equivalent functionality.
- Static placeholder sites or applications with no meaningful backend/data functionality are not suitable.

3. Build & Secure — 60 Minutes

- The official build phase is **60 minutes**.
- Teams use this time for final configuration, security hardening, testing, Docker packaging, and required documentation.
- No specific programming language, framework, frontend/backend stack, security tool, or AI tool is prescribed.
- Teams are responsible for making their own application functional and reasonably defensible.

4. Live / Remote Database Requirement

- The application must use a **live remote/shared database or backend data service**.
- A database that exists only locally inside the Docker package is **not sufficient**.
- Examples include **Firebase**, **Supabase**, or another suitable remotely accessible service.
- The application must be capable of working on another participant's laptop after the supplied setup is completed.
- Teams are responsible for required database configuration, access permissions, environment variables, and service connectivity.
- Sensitive credentials and secrets must be handled responsibly.

5. Docker Package & Setup Guide

- Each team must submit a working Docker package as a **.tar** file.
- The submission must also contain a **.txt setup guide** written during the build phase.
- The guide must explain how another team can load, configure, start, and use the application on its laptop.
- Include required commands, ports, environment variables/configuration, remote database requirements, dependencies, startup steps, and any test/default accounts where applicable.
- The guide must be practical and sufficiently clear for an opposing team to deploy the application without requiring the original developers to explain every step.

6. Submission & Random Opponent Assignment

- The **.tar package + setup guide** must be submitted before the announced build-phase deadline.
- Organizers do **not** pre-audit, security-test, or validate individual applications for vulnerabilities before the attack phase.
- After submission, team/application names are placed into a random draw.
- One representative from each team draws a chit to determine the opponent/application assigned to that team.
- The corresponding application package is transferred to the attacking team's laptop through the designated organizer process.
- All teams start the attack phase only when the official start is announced.

7. Deployment on the Opponent's Laptop

- The attacking team follows the supplied setup guide and brings the assigned application live on its own laptop.
- If a genuine deployment/runtime problem prevents startup, the attacking team may contact a member of the builder team for operational assistance.
- Such assistance is limited to setup, startup, dependencies, environment configuration, Docker operation, or basic deployment issues.
- Builder teams must not disclose vulnerabilities, hidden endpoints, attack paths, security weaknesses, or other attack information.

8. Attack — 60 Minutes

- Teams receive **60 minutes** to assess and attack the assigned live application.
- Treat the target as an unfamiliar external application: identify its technology, map functionality and endpoints, inspect inputs, and investigate weaknesses.
- Reconnaissance, penetration-testing tools, scanners, scripts, browser developer tools, and AI-assisted security workflows may be used.
- A valid vulnerability may receive credit even when the team cannot fully escalate it.

9. Reconnaissance & Vulnerability Assessment

Judges may evaluate the quality of the team's process, including:

- technology/stack discovery and fingerprinting;
- endpoint and attack-surface discovery;
- authentication, authorization, session, input-validation, API and file-handling tests;

- business-logic and data-handling analysis;
- identification of smaller weaknesses and partial findings;
- technical understanding, reasoning, and quality of evidence.

10. Attack Chains

- Teams are strongly encouraged to connect multiple weaknesses into meaningful **attack chains**.
- A chain may demonstrate how one weakness enables access, information gathering, privilege escalation, sensitive actions, data compromise, or another higher-impact result.
- Attack chains are a **high-value scoring opportunity**, but teams are not required to achieve one to score well.
- A legitimate standalone vulnerability can still earn credit without successful escalation.

11. Successful Takedown

- A takedown means a legitimate application-layer exploit causes meaningful **service disruption** or prevents intended application functionality.
- The result must be caused by the team's attack against the application and must be demonstrable and reproducible.
- A normal 403/404, ordinary error page, failed request, or unrelated runtime issue is **not** a takedown.
- DDoS, volumetric flooding, network-level denial of service, host attacks, and infrastructure abuse are prohibited.
- A takedown is a major scoring opportunity, but the overall competition is **not based on takedown alone**.

12. Attack Boundaries

Allowed: testing the assigned live web application and its intended application-level interfaces.

Prohibited: attacking the host operating system, Docker host, container runtime, event infrastructure, other teams' systems, unrelated public services, network infrastructure, or physical equipment; container escape; DDoS/volumetric resource exhaustion; and unnecessary destructive actions.

13. POC & Attack Documentation

- Teams must record important findings and attack activity as they work.
- A structured **Word document or PDF** must be submitted as the attack record.
- Each significant finding should include: vulnerability, location/endpoint, discovery method, reproduction steps, technical explanation, impact, evidence/screenshots or logs, and attack-chain relationship where applicable.
- Teams may keep a copy of their POC/evidence material for their **final presentation on the following day**.
- Documentation must be detailed enough for judges to reproduce and verify legitimate findings.

14. Scoring

Judges determine the final point allocation and scoring scale. The scoring is intended to reward both **breadth and depth**, rather than only the final takedown.

Expected scoring areas include:

- reconnaissance and technology discovery;
- endpoint and attack-surface mapping;
- valid individual vulnerabilities and partial findings;
- technical understanding and exploitation quality;
- attack methodology and reasoning;
- attack chains and escalation;
- impact and legitimate application takedown;
- evidence, reproducibility, and documentation;
- defensive/security measures demonstrated by the team.

A team without a takedown can still score strongly through high-quality reconnaissance, valid findings, analysis, methodology, attack chains, and proof.

15. Evidence Verification

- Judges may require a team to reproduce an exploit or demonstrate its evidence.
- Unsupported claims receive no credit.
- Findings based on prohibited techniques, unrelated infrastructure issues, accidental failures, or non-reproducible behavior may be rejected.
- Judges' final scoring decisions are authoritative.

16. Final Presentation — 5 Minutes / Team

Teams should present a concise technical story covering: application architecture and security measures; opponent reconnaissance; attack-surface discovery; vulnerabilities; attack methodology; attack chains; successful attacks/takedown; important partial or unsuccessful attempts; POC/evidence; and defensive lessons or improvements.

17. Fair Play & Technical Problems

- Genuine technical problems must be raised promptly with organizers.
- False evidence, fabricated vulnerabilities, deliberate misconfiguration, collusion to create a result, or abuse of prohibited systems may result in penalties or disqualification.

- Teams must not share attack findings with their opponent during the attack phase except where genuine deployment assistance is required.
- Official organizer instructions and time announcements must be followed.

18. Time Limits

- Build & Secure: **60 minutes**.
- Attack: **60 minutes**.
- Final presentation: **5 minutes per team**.
- Required submissions must be completed before the announced deadlines.

19. Judges & Organizer Authority

Judges determine scoring, verify evidence, decide whether exploits qualify, and resolve technical or rule disputes. Organizers may clarify procedures during the event where necessary to preserve fairness, safety, and smooth operation.

CORE PRINCIPLE

CYBER WAR ROOM is not a race to find one flag. It tests whether a team can build a functional application, secure it, understand an unfamiliar target, discover weaknesses, exploit them responsibly, connect findings into meaningful attack chains, prove the result, and communicate the technical story clearly.